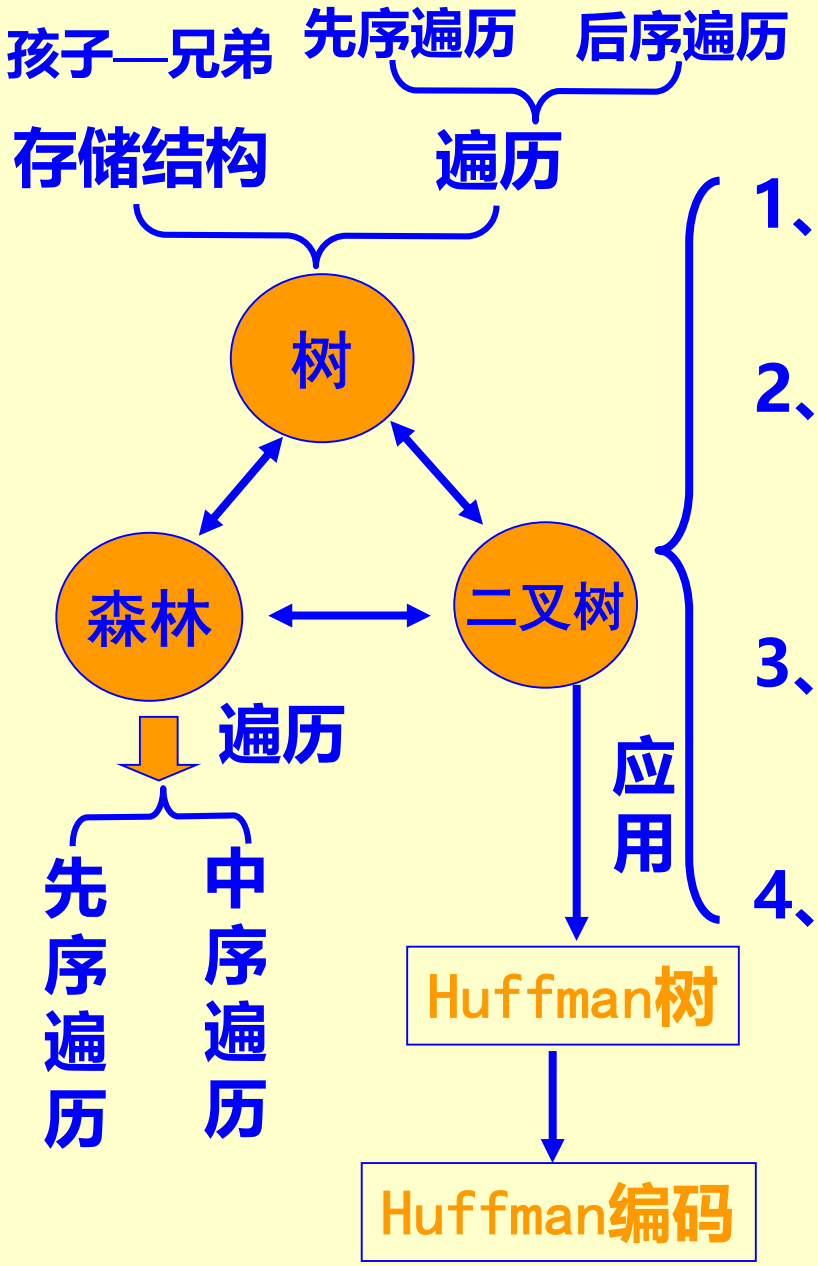
第6章 小结



1、定义和性质 1:2， 性质有3+2条

2、存储结构
- 顺序结构
- 链式结构
  - 二叉链表
  - 三叉链表

3、遍历
- 先序遍历
- 中序遍历
- 后序遍历
- 层次遍历

4、线索化 线索树
- 先序线索树
- 中序线索树
- 后序线索树

第6章结束

数据结构课程的内容

- 数据结构
  - 逻辑结构
    - 线性结构（线性表、栈　、队、串、数组）
    - 非线性结构
      - 树结构
      - 图结构
        - 多对多（m:n）
  - 物理（存储）结构
    - 顺序结构
    - 链式结构
    - 索引结构
    - 散列结构
  - 数据运算
    - 插入运算
    - 删除运算
    - 修改运算
    - 查找运算
    - 排序运算

内 容 安 排

章	内 容	学时	章	内 容	学时
1	绪 论	2	7	图	5
2	线性表	5	8	动态存储管理	略
3	栈和队列	3	9	查找	3
4	串	3	10	内部排序	5
5	数组和广义表	3	11	外部排序	略
6	树和二叉树	7	12	文件	略

第7章 图

7.1 基本术语

7.2 存储结构

7.3 图的遍历

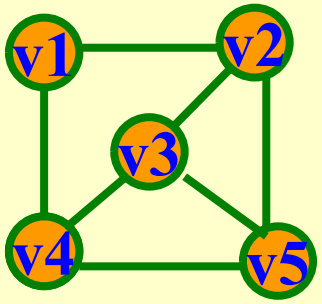
7.4 图的连通性

7.5 图的应用

7.1 图的基本术语

图：记为 G＝(V, E)

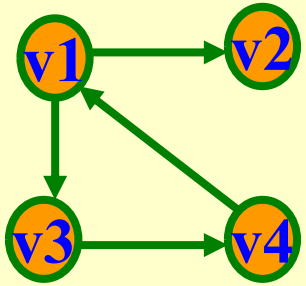
V=vertex
E=edge



其中：V是G的顶点集合，是有穷非空集；

E是G的边集合，是有穷集。

问：当E(G)为空时，图G存在否?
答：还存在！但此时图G只有顶点而没有边。



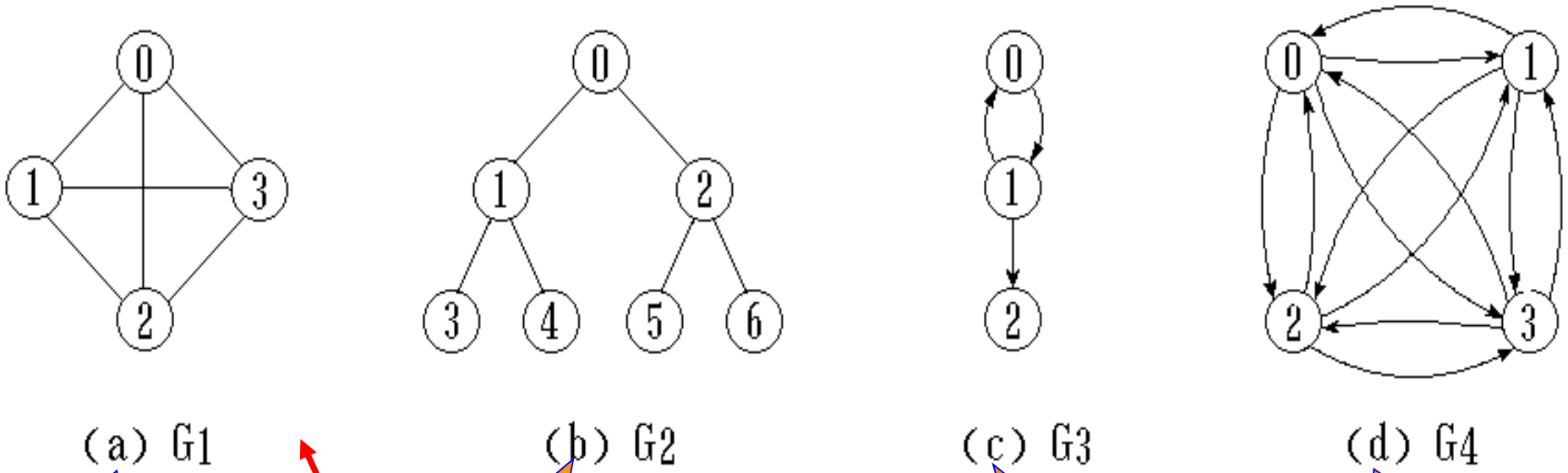
有向图：图G中的每条边都是有方向的；

无向图：图G中的每条边都是无方向的；

完全图：图G任意两个顶点都有一条边相连接；

- 若 n 个顶点的无向图有 $n(n-1)/2$ 条边, 称为无向完全图
- 若 n 个顶点的有向图有$n(n-1)$ 条边, 称为有向完全图

例：判断下列4种图形各属什么类型？



无向完全图

无向图(树)

有向图

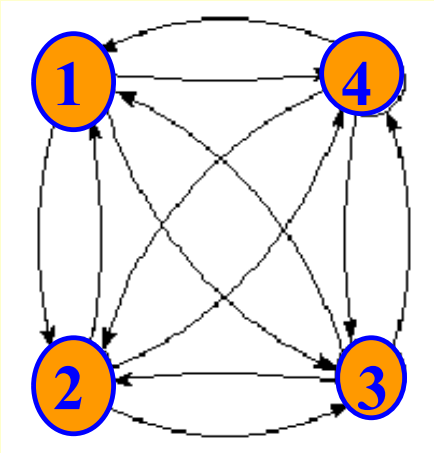
有向完全图

$n(n-1)/2$ 条边

$n(n-1)$ 条边

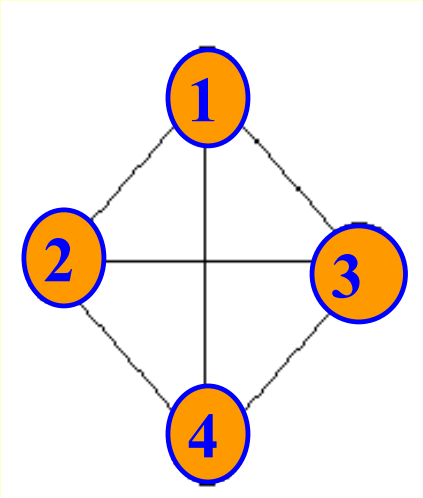
G1的顶点集合为V(G1)={0,1,2,3}
边集合为E(G1)={(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)}

证明：



① 完全有向图有$n(n-1)$条边。

证明：若是完全有向图，则n个顶点中的每个顶点都有一条弧指向其它n-1个顶点，因此总边数=$n(n-1)$



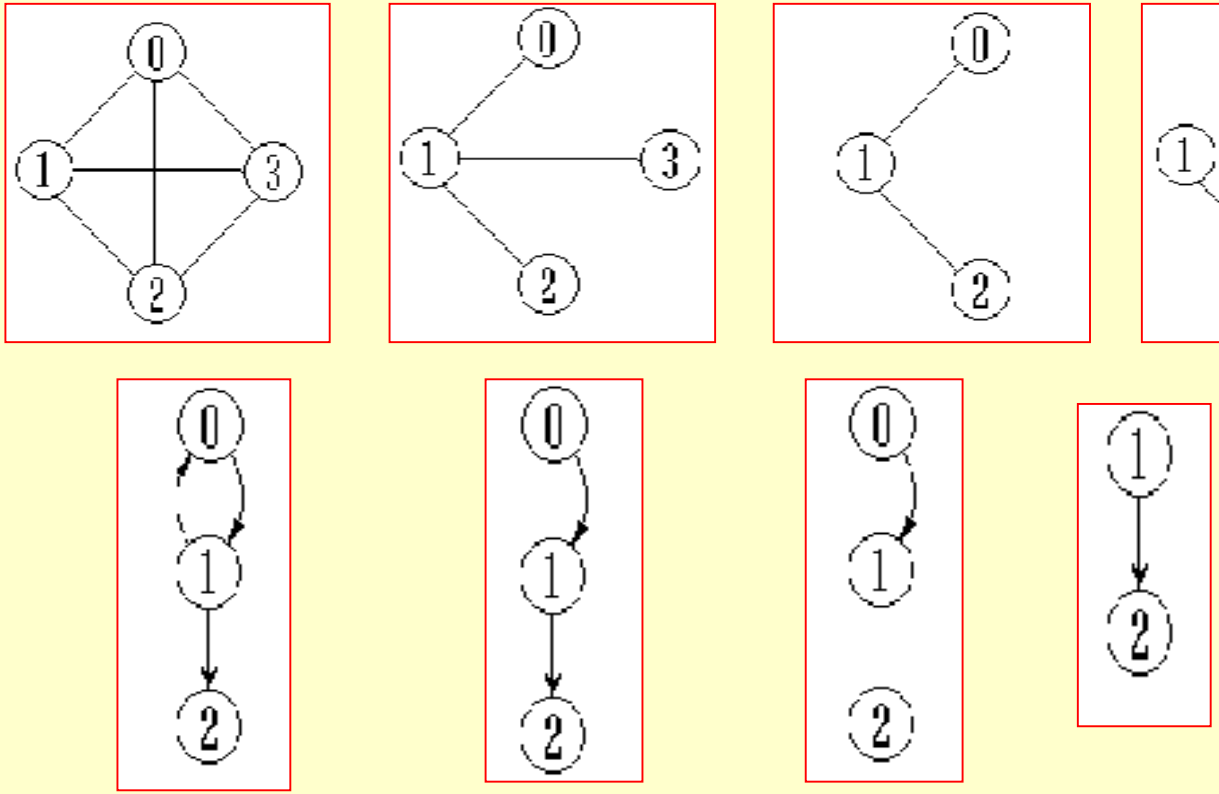
② 完全无向图有$n(n-1)/2$ 条边。

证明：从①可以直接推论出无向完全图的边数——因为无方向，两弧合并为一边，所以边数减半，总边数为$n(n-1)/2$。

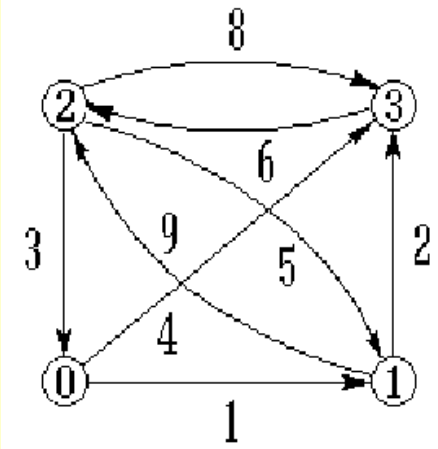
稀疏图：边较少的图。通常边数远少于nlogn

稠密图：边很多的图。
- 无向图中，边数接近$n(n-1)/2$
- 有向图中，边数接近$n(n-1)$

子　图：设有两个图 $G = (V, E)$ 和 $G' = (V', E')$。若 $V' \subseteq V$ 且 $E' \subseteq E$, 则称 图G' 是 图G 的子图。



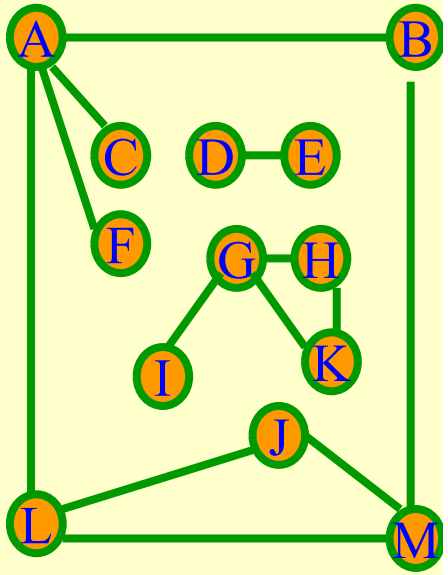
带权图：即边上带权的图。其中权是指每条边可以标上具有某种含义的数值（即与边相关的数）。



网　络：→带权图

连通图：在无向图中，若从顶点v_1到顶点v_2有路径，则称顶点v_1与v_2是连通的。如果图中任意一对顶点都是连通的，则称此图是连通图。

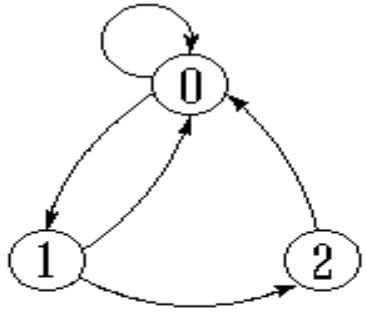
非连通图的极大连通子图叫做连通分量。



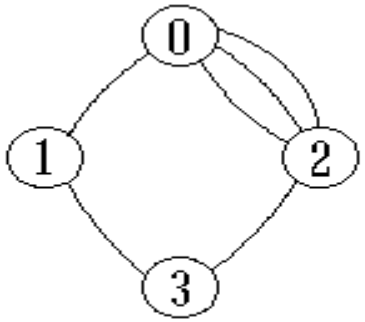
强连通图：在有向图中，若对于每一对顶点v_i和v_j，都存在一条从v_i到v_j和从v_j到v_i的路径，则称此图是强连通图。

非强连通图的极大强连通子图叫做强连通分量。

有两类图形不在本章讨论之列：

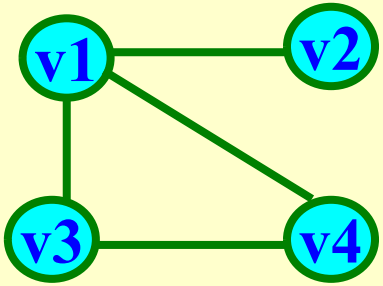


（a）带自身环的图



（b）多重图

图的基本术语(续)



生成树：是一个极小连通子图，它含有图中全部n个顶点，但只有*n*-1条边。

- **如果在生成树上添加1条边，必定构成一个环。**
- **若图中有n个顶点，却少于*n*-1条边，必为非连通图。**

生成森林：若干棵生成树的集合，含全部顶点，但构成这些树的边或弧是最少的。

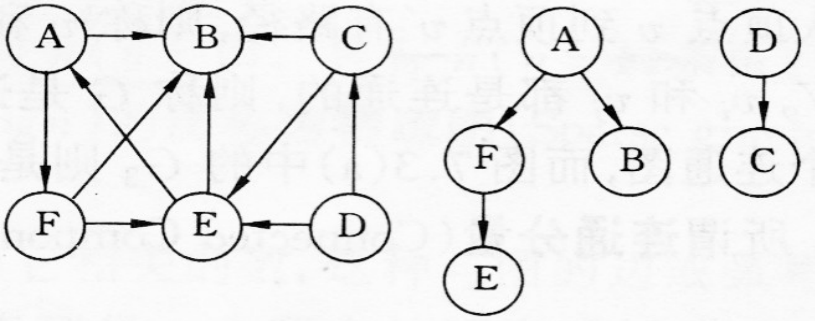
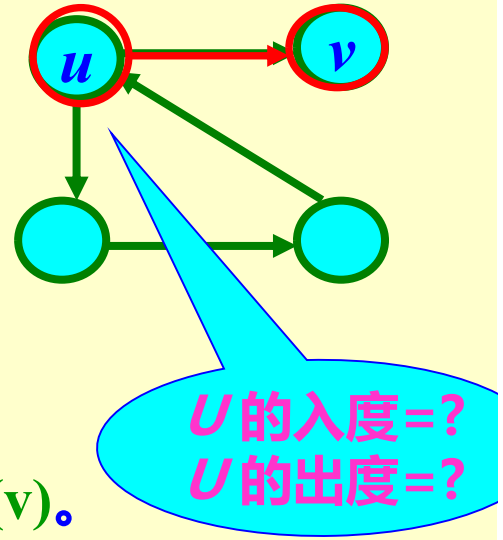


图 7.6　一个有向图及其生成森林

邻接点：若 (u, v) 是 $E(G)$ 中的一条边，则称 u 与 v 互为邻接顶点。



弧头和弧尾：有向边(u, v)称为弧，边的始点u 叫弧尾，终点v 叫弧头。

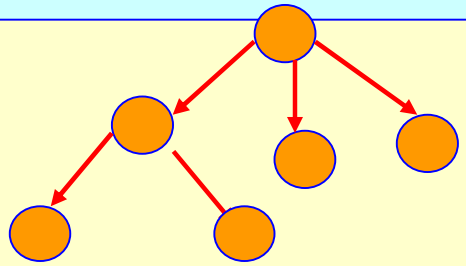
度：顶点v 的度是与它相关联的边的条数。记作TD(v)。

在有向图中，顶点的度等于该顶点的入度与出度之和。

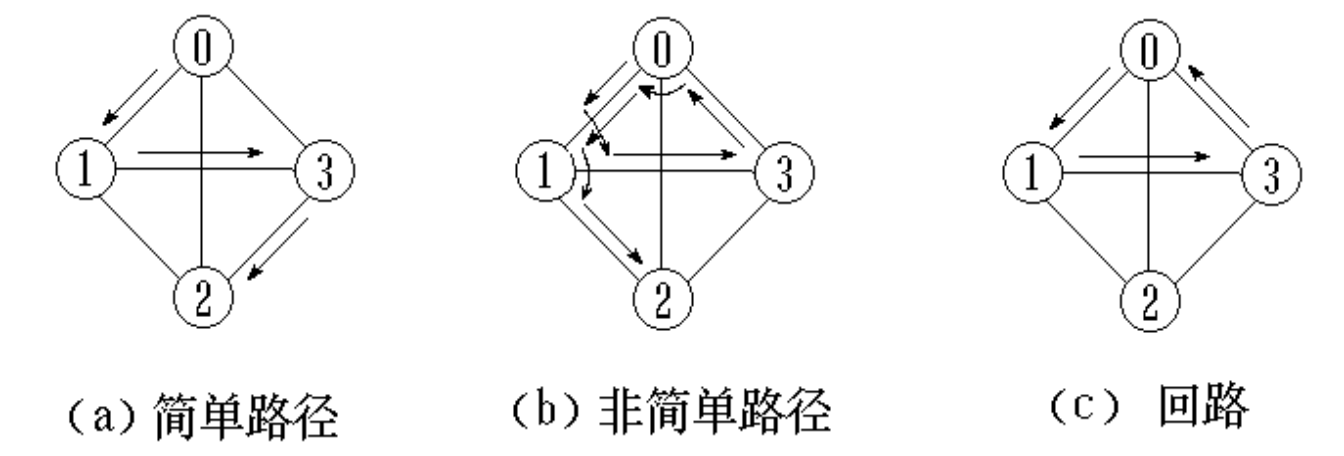
入度和出度：顶点 v 的入度是以 v 为终点的有向边的条数，记作 ID(v)；
顶点 v 的出度是以 v 为始点的有向边的条数，记作 OD(v)。

问：当有向图中仅1个顶点的入度为0,其余顶点的入度均为1，此时是何形状?

答：是树！而且是一棵有向树！



图的术语（续）



（a）简单路径　　（b）非简单路径　　（c） 回路

路径：在图 G = (*V*, *E*) 中，若从顶点 v_i 出发，沿一些边经过一些顶点 $v_{p1}, v_{p2}, \ldots, v_{pm}$，到达顶点$v_j$。则称顶点序列 (v_i v_{p1} v_{p2} … v_{pm} v_j) 为从顶点v_i到顶点 v_j 的路径。它经过的边(v_i, v_{p1})、(v_{p1}, v_{p2})、…、(v_{pm}, v_j)应当是属于E的边。

路径长度：
- 非带权图的路径长度是指此路径上边的条数；
- 带权图的路径长度是指路径上各边的权之和。

简单路径：路径上各顶点 $v_1, v_2, \ldots, v_m$ 均不互相重复。

回路：若路径上第一个顶点 v_1 与最后一个顶点v_m 重合，则称这样的路径为回路或环。

图的抽象数据类型

ADT Graph {

数据对象V： V是具有相同特性的数据元素的集合，称为顶点集。

数据关系 R： R={VR}；VR={<v,w>|v,w∈V 且 P(v,w),
<v,w>表示从v到w的弧，
谓词P(v,w)定义了弧<v,w>的意义或信息}

基本操作P：

CreatGraph (&G, V,VR);

初始条件：V是图的顶点集，VR是图中弧的集合。

操作结果：按V和VR的定义构造图G。

InsertVex (&G, *v*);

注意：V 的大小写含义不同！

初始条件：图G存在，*v*和图中顶点有相同特征。

操作结果：在图G中添加新顶点。

…… (参见教材P156-257)

}ADT Graph

7.2 图的存储结构

图的特点： **非线性结构（m :n)**

顺序存储结构： 难！(多个顶点，无序，仅用顶点坐标无法表达相互关系)

但可用数组描述元素间关系。

邻接矩阵

链式存储结构： **可用多重链表**

- 邻接表
- 十字链表
- 邻接多重表

1. 邻接矩阵(数组)表示法
2. 邻接表(链式)表示法
3. 十字链表表示法
4. 邻接多重表表示法

各种表示法成立的原则：
存入电脑后能惟一复原

1. 邻接矩阵（数组）表示法

① 建立一个顶点表和一个邻接矩阵。

表示各个顶点之间关系

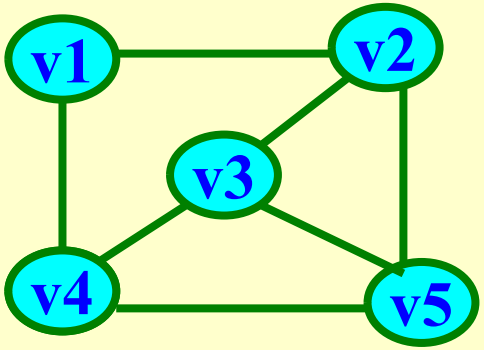
记录各个顶点信息

② 设图 A = (*V*, *E*) 有 *n* 个顶点，则图的邻接矩阵是一个二维数组 A.*Edge*[*n*][*n*]，定义为：

$$A.Edge[i][j] = \begin{cases} 1, & \text{如果} \ <i, j> \in E \ \text{或者} \ (i, j) \in E \\ 0, & \text{否则} \end{cases}$$

例1：下面无向图的邻接矩阵如何表示？

A



顶点表：（v1 v2 v3 v4 v5 ）

邻接矩阵：

$$A.Edge = \begin{array}{c} v1 \\ v2 \\ v3 \\ v4 \\ v5 \end{array} \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

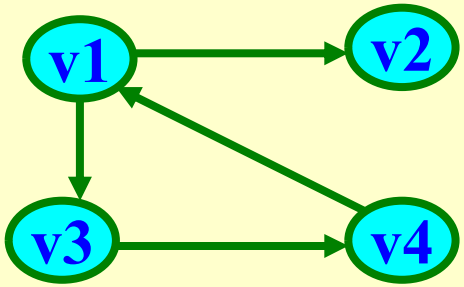
分析1：无向图的邻接矩阵是对称的；

分析2：顶点*i* 的度 = 第 *i* 行 (列) 中1 的个数；

特别：完全图的邻接矩阵中，对角元素为0，其余全1。

例2： 下面有向图的邻接矩阵如何表示？

A



顶点表： (v1 v2 v3 v4)

邻接矩阵：

$$A.Edge = \begin{array}{c} v1 \\ v2 \\ v3 \\ v4 \end{array} \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

注：在有向图的邻接矩阵中，

第i行含义：以结点v_i为尾的弧(即出度边)；

第i列含义：以结点v_i为头的弧(即入度边)。

分析1：有向图的邻接矩阵可能是不对称的。

分析2：顶点v_i的出度=第i行元素之和， $OD(V_i)=\sum A.Edge[i][j]$

顶点v_i的入度=第i列元素之和。 $ID(V_i)=\sum A.Edge[j][i]$

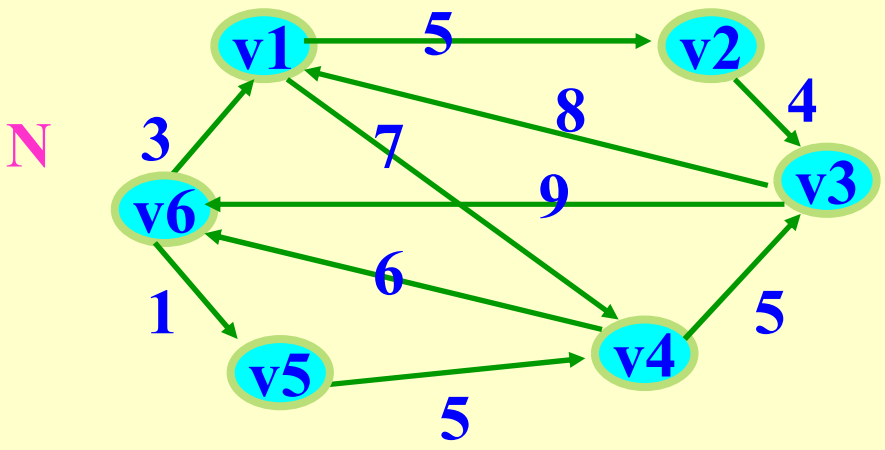
顶点的度=第i行元素之和+第i列元素之和，

即： $TD(v_i) = OD(vi) + ID(vi)$

例3： 有权图（即网络）的邻接矩阵如何表示？

定义： $A.Edge[i][j] = \begin{cases} W_{ij} & <vi, vj> \text{或}(vi, vj) \in VR \\ \infty & \text{无边（弧）} \end{cases}$

以有向网为例：



顶点表： (v1 v2 v3 v4 v5 v6)

邻接矩阵：

$$N.Edge = \begin{array}{c} v1 \\ v2 \\ v3 \\ v4 \\ v5 \\ v6 \end{array} \begin{pmatrix} \infty & 5 & \infty & 7 & \infty & \infty \\ \infty & \infty & 4 & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty & \infty & 9 \\ \infty & \infty & 5 & \infty & \infty & 6 \\ \infty & \infty & \infty & 5 & \infty & \infty \\ 3 & \infty & \infty & \infty & 1 & \infty \end{pmatrix}$$

邻接矩阵法优点： 容易实现图的操作，如：求某顶点的度、判断顶点之间是否有边（弧）、找顶点的邻接点等等。

邻接矩阵法缺点： n个顶点需要*n*n*个单元存储边(弧);空间效率为O(n^2)。

对稀疏图而言尤其浪费空间。

图的邻接矩阵在机内如何表示？ （参见教材P161）

注：用两个数组分别存储顶点表和邻接矩阵

```
#define INFINITY  INT_MAX            //最大值∞
#define MAX_VERTEX_NUM     20  //假设的最大顶点数

Typedef  enum {DG, DN, AG, AN } GraphKind; //有向/无向图，有向/无向网
```

```
Typedef   struct   ArcCell{          //弧（边）结点的定义
 VRType     adj;        //顶点间关系，无权图取1或0；有权图取权值类型
  InfoType   *info;   //该弧相关信息的指针
}ArcCell, AdjMatrix [ MAX_VERTEX_NUM ] [MAX_VERTEX_NUM ];
```

```
Typedef struct{                          //图的定义
VertexType vexs [MAX_VERTEX_NUM ] ;  //顶点表，用一维向量即可(n)
AdjMatrix arcs;              //邻接矩阵n*n
Int   Vernum,   arcnum;  //顶点总数n，弧（边）总数e
GraphKind kind;   //图的种类标志
}Mgraph;
```

对于n个顶点的图或网，空间效率=O(n^2)

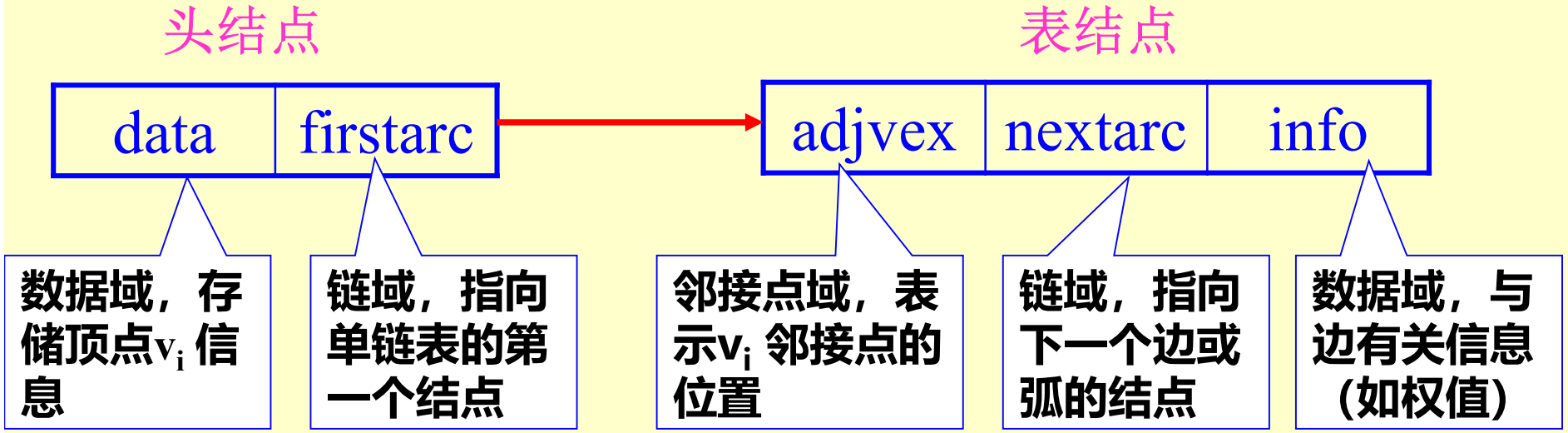
例：用邻接矩阵生成无向网的算法（参见教材P162）

```
Status CreateUDN(Mgraph &G){          //无向网的构造，用邻接矩阵表示
scanf(&G.vexnum, &G.arcnum, &IncInfo); //输入总顶点数n、总弧数e和信息
for(i=0;i<G.vexnum,;++i) scanf(&G.vexs[i] );//输入n个顶点值，存入一维向量

for(i=0; i<G.vexnum; ++i)  //对邻接矩阵n*n个单元初始化，adj=∞,info=NULL
for(j=0;j<G.vexnum;++j) G.arcs[i][j]={INFINITY, NULL};

 for(k=0;k<G.arcnum;++k){  //给邻接矩阵有关单元赋初值(循环次数 = 弧数e
 scanf(&v1, &v2, &w); //输入弧的两顶点以及对应权值
 i=LocateVex(G,v1); j=LocateVex(G,v2); //找到两顶点在矩阵中的位置(n次)
 G.arcs[i][j].adj=w; //输入对应权值
 If(IncInfo) Input(*G.arcs[i][j].info); //如果弧有信息则填入
 G.arcs[i][j] = G.arcs [j] [i]；  //无向网是对称矩阵
 }
 return OK;
} // CreateUDN
```

对于n个顶点e条弧的网，
建网时间效率 =
$O(n+n^2+e*n)$

2. 邻接表（链式）表示法

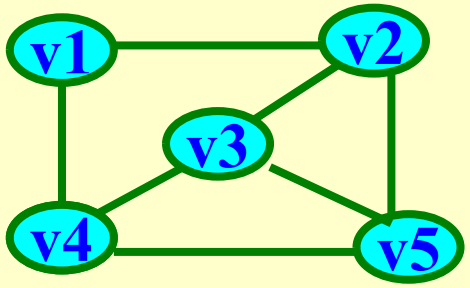
① 对每个顶点v_i建立一个单链表，把与v_i有关联的边的信息（即度或出度边）链接起来，表中每个结点都设为3个域:

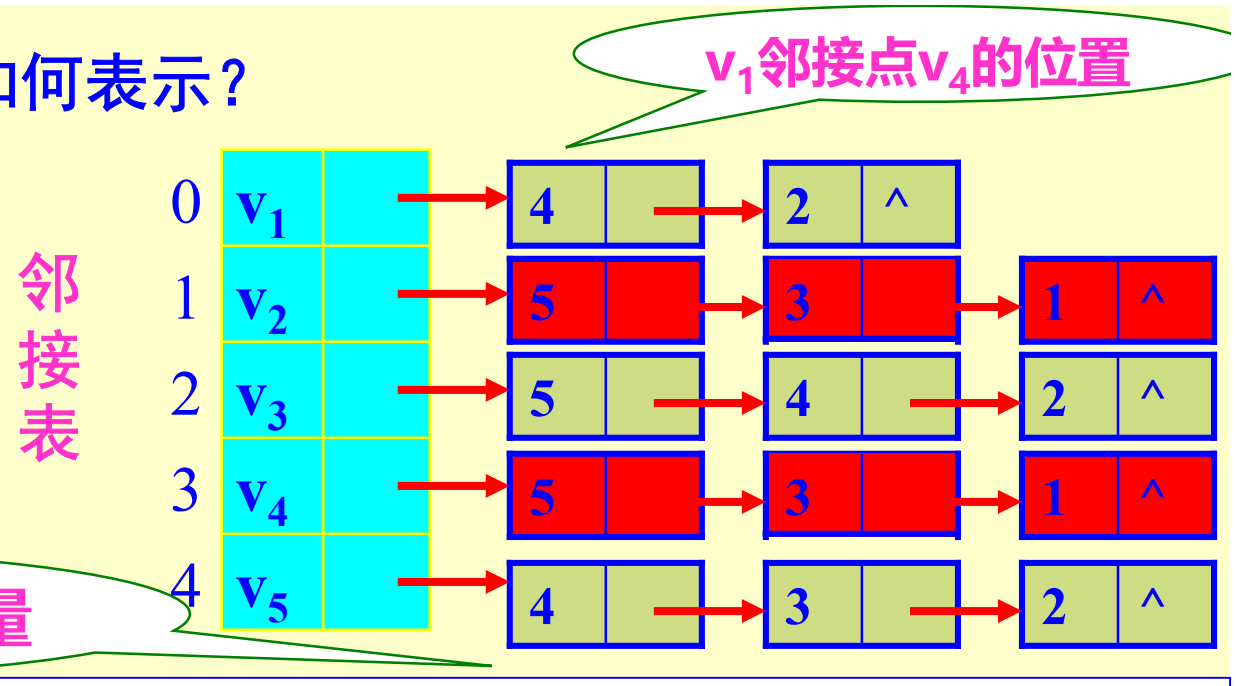


② 每个单链表还应当附设一个头结点（设为2个域），存v_i信息;

③ 每个单链表的头结点另外用顺序存储结构存储。

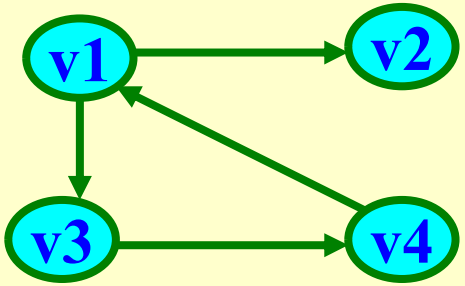
例1：无向图的邻接表如何表示？

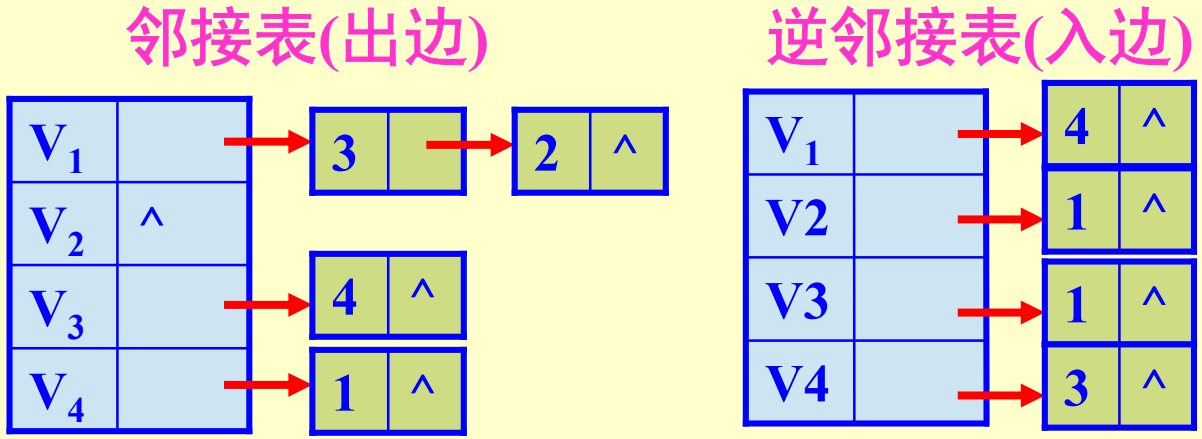




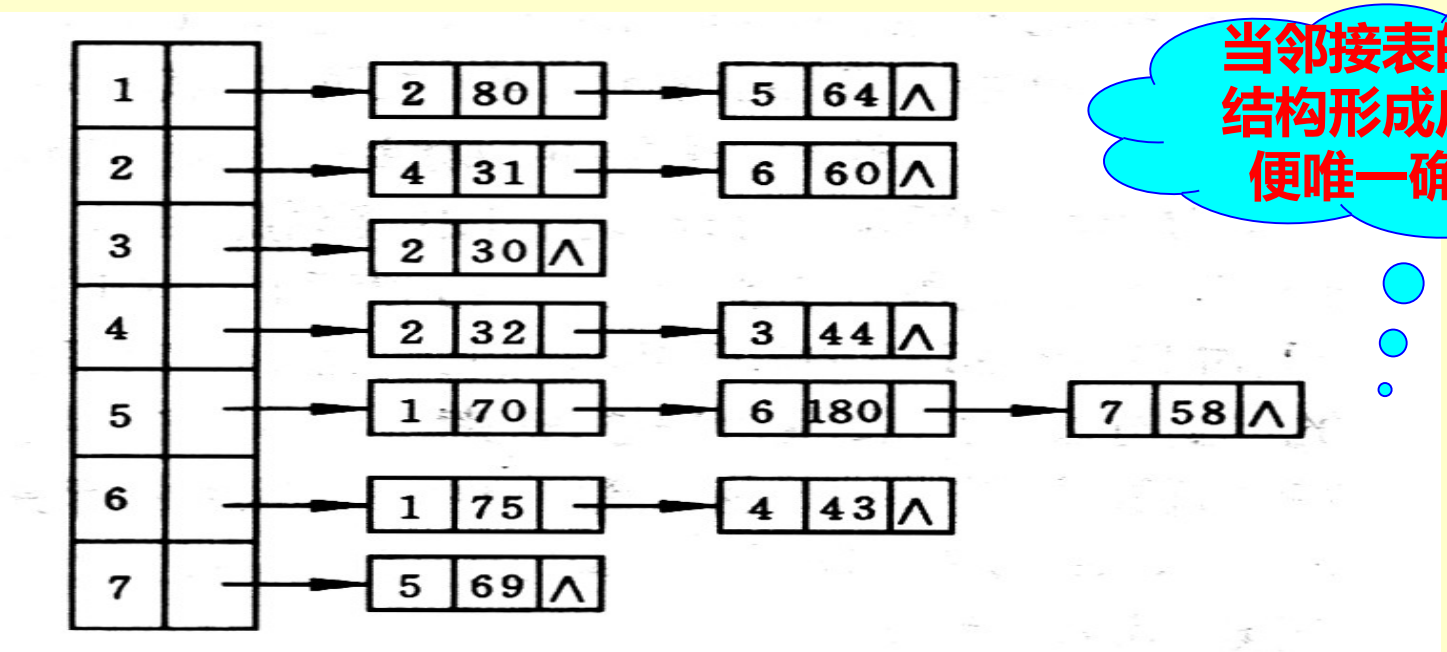
请注意：邻接表不惟一！因各个边结点的链入顺序是任意的。

例2：有向图的邻接表如何表示？

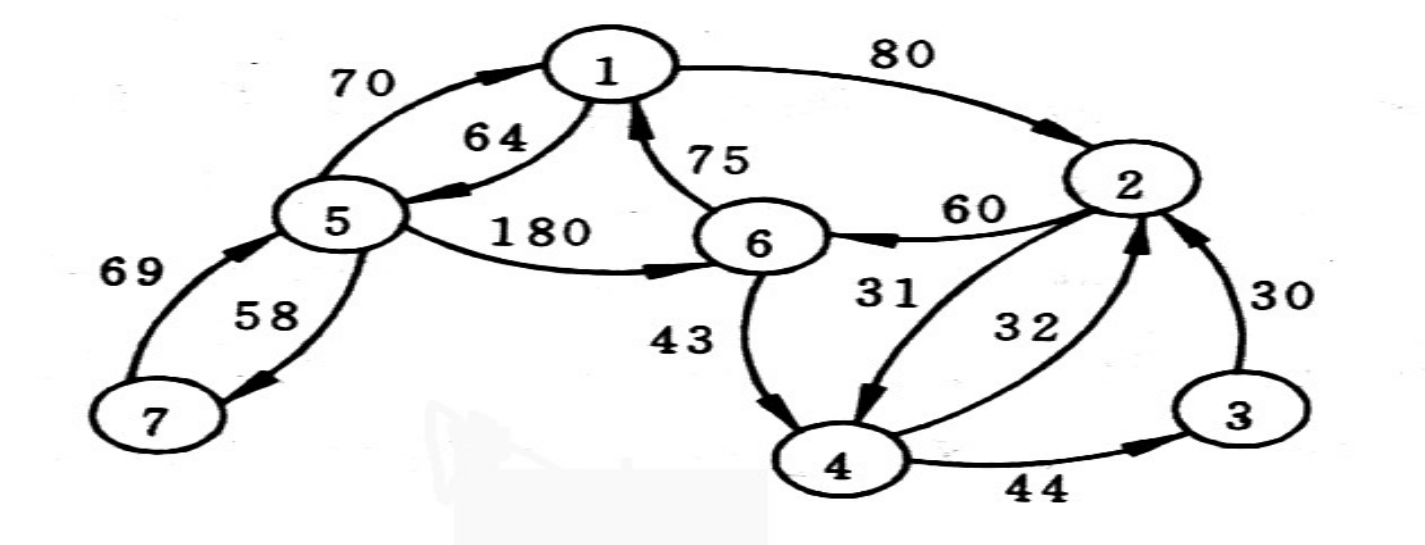




例3：已知某网的邻接（出边）表，请画出该网络。



当邻接表的存储结构形成后，图便唯一确定！



邻接表存储法的特点：—它其实是对邻接矩阵法的一种改进

分析1：对于n个顶点e条边的无向图，邻接表中除了n个头结点外，只有2e个表结点,空间效率为O(n+2e)。

若是稀疏图($e<<n^2$)，则比邻接矩阵表示法$O(n^2)$省空间。

怎样计算无向图顶点的度？ *TD(Vi)*=单链表中链接的结点个数

分析2:在有向图中，邻接表中除了n个头结点外，只有e个表结点,空间效率为O(n+e)。若是稀疏图，则比邻接矩阵表示法合适。

怎样计算有向图顶点的出度？OD(Vi)＝单链出边表中链接的结点数

怎样计算有向图顶点的入度？I D(Vi) ＝邻接点为Vi的弧个数

需遍历全表

怎样计算有向图顶点Vi的度：$TD(Vi) = OD(Vi) + ID(Vi)$

邻接表的优点： 空间效率高；容易寻找顶点的邻接点；

邻接表的缺点： 判断两顶点间是否有边或弧，需搜索两结点对应的单链表，没有邻接矩阵方便。

讨论：邻接表与邻接矩阵有什么异同之处？

1. 联系：**邻接表中每个链表对应于邻接矩阵中的一行，链表中结点个数等于一行中非零元素的个数。**

2. 区别：

① 对于任一确定的无向图，邻接矩阵是唯一的（行列号与顶点编号一致），但邻接表不唯一（链接次序与顶点编号无关）。

② 邻接矩阵的空间复杂度为$O(n^2)$,而邻接表的空间复杂度为$O(n+e)$。

3. 用途：

邻接矩阵多用于稠密图的存储（e接近$n(n-1)/2$）；

而邻接表多用于稀疏图的存储（$e<<n^2$）

图的邻接表在机内如何表示？ (参见教材P163)

```
#define MAX_VERTEX_NUM 20  //假设的最大顶点数
Typedef  struct  ArcNode {     //弧结构
    int  adjvex;                //该弧所指向的顶点位置
    struct  ArcNode *nextarcs; //指向下一条弧的指针
    InfoArc    *info;          //该弧相关信息的指针
} ArcNode;
```

图的邻接表生成算法作为自测题

```
Typedef   struct  VNode{      //顶点结构
    VertexType   data;        //顶点信息
    ArcNode   * firstarc;     //指向依附该顶点的第一条弧的指针
}VNode, AdjList[  MAX_VERTEX_NUM ];
```

```
Typedef   struct {            //图结构
    AdjList   vertics ;       //应包含邻接表
    int  vexnum, arcnum; //应包含顶点总数和弧总数
    int  kind;                //还应说明图的种类（用标志）
}ALGraph;
```

空间效率为O(n+2e)或O(n+e)

时间效率为O(n+e*n)

3. 十字链表表示法（不考）

它是有向图的另一种链式结构。

思路：**将邻接矩阵用链表存储，是邻接表、逆邻接表的结合。**

1、开设弧结点，设5个域（**每段弧是一个数据元素**）

2、开设顶点结点，设3个域（**每个顶点也是一个数据元素**）

弧结点

tailvex	headvex	hlink	tlink	info

tailvex: 弧尾顶点位置
headvex: 弧头顶点位置
hlink: 弧头相同的下一弧位置
tlink: 弧尾相同的下一弧位置
info: 弧信息

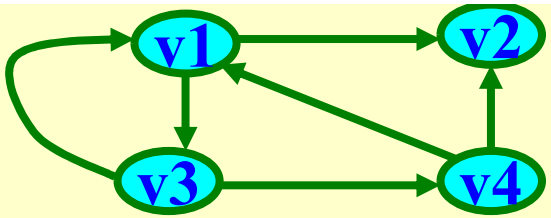
顶点结点

data	Firstin	Firstout

data ：顶点信息
Firstin ：以顶点为弧头的第一条弧结点
Firstout: 以顶点为弧尾的第一条弧结点

n个顶点的集合怎样储存？ **仍用顺序存储结构**

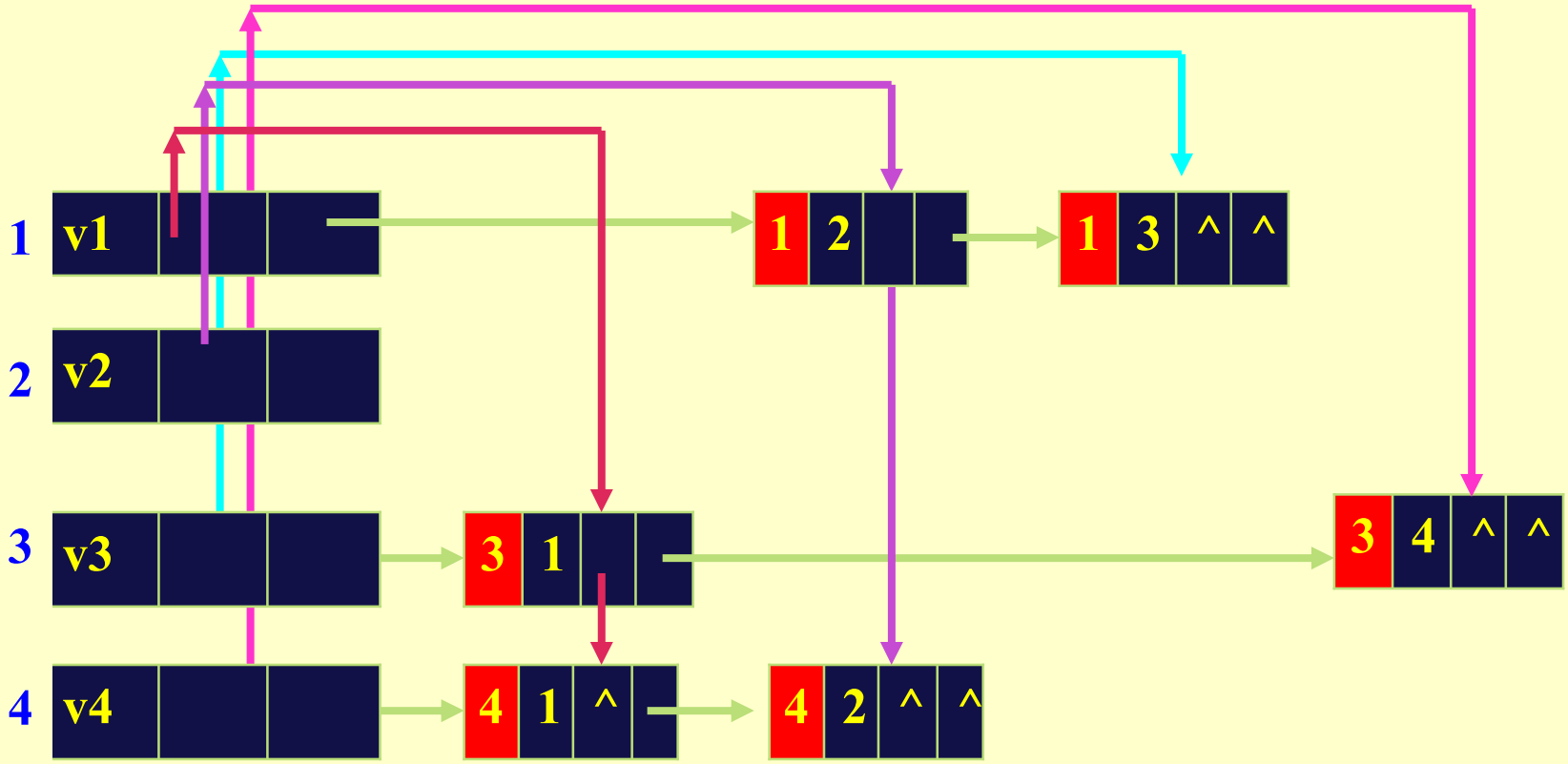
例：画出有向图的十字链表。



顶点结点	data	Firstin	Firstout

弧结点	tailvex	headvex	hlink	tlink

此无权图未开第4分量



十字链表优点：容易操作，如求顶点的入度、出度等。

空间复杂度和建表的时间复杂度都与邻接表相同。

十字链表存储结构描述：

```
#define MAX_VERTEX_NUM 20
```

```
Typedef  struct  ArcBox {     //弧结点结构，5分量
   int  tailvex , headvex ;
   struct  ArcBox * hlink , tlink;
   InfoType    *info;
} ArcBox;
```

```
Typedef   struct  VexNode{  //顶点结构, 3分量
    VertexType   data;
     ArcBox   * firstin,*firstout;
}VexNode;
```

```
Typedef   struct {              //图结构,整体概念
      VexNode xlist[  MAX_VERTEX_NUM ];   //表头向量
      int  vexnum, arcnum;
}OLGraph;
```

4. 邻接多重表表示法（不考）

这是无向图的另一种存储结构，当对边操作时建议采用此种结构存储。

1、设立边结点， 6个域（每条边是一个数据元素）

2、设立顶点结点， 2个域（每个顶点也是一个数据元素）

边结点

mark	ivex	ilink	jvex	jlink	info

mark：标志域
ivex， jvex：边依附的两个顶点位置
ilink: 指向下一条依附顶点 i 的边位置
jlink; 指向下一条依附顶点 j 的边位置
info: 边信息

顶点结点

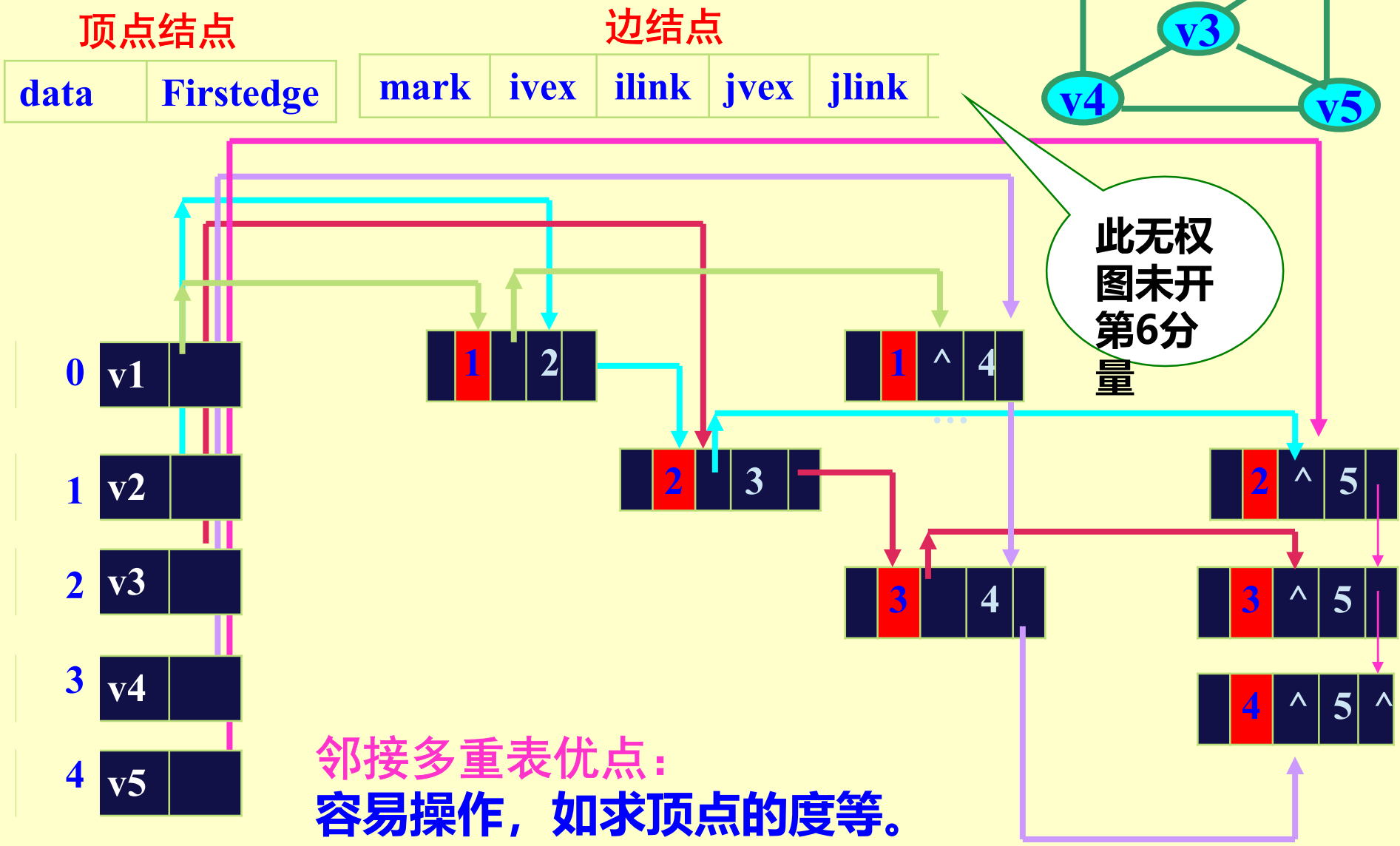
data	Firstedge

data ：存储顶点信息
Firstedge：依附顶点的第一条边结点

n个顶点的集合怎样储存？

仍用顺序存储结构

例：画出无向图的邻接多重表



邻接多重表优点：

容易操作，如求顶点的度等。

空间复杂度和建表的时间复杂度都与邻接表相同。